

Terraform Mastery Documentation

Comprehensive Engineering Notes & Step-by-Step Practical Implementation Guide

Author: Vikas Rawat | Systems & Cloud Infrastructure | Date: May 25, 2026

1. Introduction to Infrastructure as Code (IaC)

Before exploring the files and commands, it is essential to look at what Terraform is doing under the hood. Traditionally, building infrastructure required logging into web consoles (like the AWS Management Console) and clicking buttons to launch servers, modify networks, or provision databases. This approach is error-prone, slow, and impossible to track or replicate reliably.

Terraform completely transforms this paradigm by introducing **Infrastructure as Code (IaC)**. Instead of performing manual UI steps, you describe your entire infrastructure network architecture using a simple, human-readable language called **HCL (HashiCorp Configuration Language)**. Terraform reads these text configuration files, builds an implicit dependency graph, interacts with cloud APIs, and deploys your systems identically every time.

The Core Value Proposition: By moving your infrastructure declaration into text documents, you treat hardware exactly like software. This means you can version control your servers using Git, rollback changes instantly, peer-review network settings before deployment, and completely stand up or tear down complex multi-region architectures using simple automated terminal scripts.

2. Complete File Directory & Workspace Breakdown

Based on your current workspace file tree structure shown in the explorer, let's look at the exact purpose and functional mechanic of every single file and folder. Understanding this anatomy ensures you no longer feel like you are just copying and pasting code blind.

File / Directory Path	Purpose & Internal Working Principles
<code>.terraform/</code>	Local Provider Cache & Module Map: This hidden folder is managed strictly by Terraform. When you run initialization routines, it downloads the compiled cloud binary engines (like the AWS provider executable) required to execute API payloads. It also tracks mappings for your local child modules.
<code>.terraform.lock.hcl</code>	Dependency Lock File: This acts as a security lock for your provider versions. It tracks the exact cryptographic hash of the provider plugin binary you downloaded during initialization, ensuring that anyone else executing your code uses the exact same software engine version.
<code>dev.tfvars</code> / <code>qa.tfvars</code>	Environment Variable Values: These files store actual values for your variables matching specific targets (e.g., setting instance types to <code>t2.micro</code> in Dev, but scaling to larger sizes in QA). They keep your primary code architecture clean and parameter-driven.
<code>main.tf</code> (Root)	Primary Execution Entrypoint: The top-level orchestrator. In your current workflow, this file coordinates your core custom infrastructure components—invoking your local network modules, declaring security firewalls, and provisioning your main compute instances.
<code>providers.tf</code>	Cloud Authentication & Backend State Matrix: Tells Terraform exactly which cloud provider to interface with, maps the default execution region (<code>ap-south-1</code> Mumbai), and hosts the cloud state remote S3 storage configuration.
<code>variables.tf</code>	Input Declarations: The input definition schema. It explicitly sets up the configurable knobs for your templates, allowing your files to accept external inputs instead of having hardcoded parameters inside the resource declarations.
<code>rds.tf</code>	Isolated Database Architecture: Contains your managed MySQL backend resource blocks and dynamic database subnet group structures, kept in a separate file for clear segregation of duties.
<code>containers.tf</code>	Container Registries & Container Orchestration: Houses the ECR Docker registry blocks, the dedicated IAM instance role permissions, and the advanced multi-AZ EKS Kubernetes cluster orchestrator definitions.
<code>import_practice.tf</code>	Console Import Sandbox: A specialized file containing structural tracking blocks used to securely link pre-existing components built via web consoles straight back into automated state tracking.
<code>terraform.tfstate</code>	

File / Directory Path	Purpose & Internal Working Principles
	The State Database File (The Engine Brain): A highly detailed JSON mapping document tracking every real-world cloud resource ID matching your local code blocks. <i>Never edit this file manually.</i>
<code>terraform.tfstate.backup</code>	State Failover Mirror: An automated snapshot copy generated by the engine right before execution runs, preserving a recovery pathway in case an application loop gets aborted mid-flight.
<code>modules/vpc/</code>	Encapsulated Network Layer Module: A fully contained reusable child directory hosting isolated sub-blueprints (<code>main.tf</code> , <code>outputs.tf</code>) that generate networks across multiple distinct availability data centers.

Crucial Concept — How Terraform Compiles Files: To the Terraform engine, filenames inside a single folder do not matter. When you initiate an execution, it merges every single `.tf` file in that directory together into one massive single blueprint. We divide them into files purely so humans can easily navigate and read the codebase!

3. The Professional Step-by-Step Implementation Lifecycle

The core daily routine of an automation infrastructure architect centers on executing four exact core actions. This lifecycle forms the foundation of all infrastructure deployments:

Step 1: Workspace Initialization (`terraform init`)

This is the initial setup command for your directory. When triggered, the compiler inspects your configuration files, extracts your provider block mappings, communicates with public registries, and downloads the binary plugin engines (such as `terraform-provider-aws_v6.45.0_x5.exe`) into your local hidden directory cache. It also links your S3 bucket remote state communication channels.

```
terraform init
```

If you modify your backend configuration or switch state engines later, you must run an explicit reconfiguration sequence:

```
terraform init -reconfigure
```

Step 2: Dry-Run Speculative Architecture Planning (`terraform plan`)

This step acts as a safe preview phase. Terraform reaches out to live cloud environment APIs, inspects current structural baselines, references your state database tracker file, and builds an exhaustive audit log showing you exactly what it will build (`+`), modify (`~`), or wipe out (`-`) before making any real changes.

```
terraform plan
```

Step 3: Actual Real-World Deployment Execution (`terraform apply`)

This command converts text configurations into live, working environments. Terraform builds an execution graph, processes dependencies in order, sends secure requests to cloud APIs, waits for verification from the data centers, and logs real-world tracking resource metrics into your state database file.

```
terraform apply
```

Step 4: Clean Up & Complete Resource Destruction (`terraform destroy`)

This step completely tears down and deletes everything provisioned by the project. It safely removes live cloud components to keep your cloud spend at zero, while leaving your local code files untouched on your machine.

```
terraform destroy
```

4. Complete Technical Code Reference Manual

Below are the clean, verified, production-grade configurations that align exactly with all your May 21st and May 22nd assignment parameters.

4.1 Remote Backend and Provider Alignment Matrix (`providers.tf`)

This file sets up your AWS authentication parameters and switches your tracking state from your local computer to your secure cloud bucket.

```
terraform {
  required_version = ">= 1.5.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}
```

```
# Storing the infrastructure brain state in a remote cloud bucket
backend "s3" {
  bucket = "vikas-terraform-state-1991"
  key     = "assignments/dev/terraform.tfstate"
  region  = "ap-south-1" # Mumbai data center home
  encrypt = true
}

provider "aws" {
  region = "ap-south-1" # Forces consistent alignment with your VPC network location
}
```

4.2 Custom High-Availability Network Layer Module

These files live deep inside the isolated child subfolder path: `modules/vpc/`.

Module Logical Network Declaration (`modules/vpc/main.tf`):

```
resource "aws_vpc" "custom_network" {
  cidr_block      = "10.0.0.0/16"
  enable_dns_hostnames = true

  tags = {
    Name           = "vikas-modular-vpc"
    ManagedBy      = "Terraform"
    Environment     = "Dev"
  }
}

# Subnet A: Placed in Availability Zone 1a
resource "aws_subnet" "public_subnet" {
  vpc_id            = aws_vpc.custom_network.id
  cidr_block        = "10.0.1.0/24"
  availability_zone  = "ap-south-1a"
  map_public_ip_on_launch = true

  tags = { Name = "vikas-public-subnet-1a" }
}

# Subnet B: Placed in Availability Zone 1b (Required for EKS High-Availability Rules!)
resource "aws_subnet" "public_subnet_b" {
  vpc_id            = aws_vpc.custom_network.id
  cidr_block        = "10.0.2.0/24"
  availability_zone  = "ap-south-1b"
  map_public_ip_on_launch = true

  tags = { Name = "vikas-public-subnet-1b" }
}
```

Module Security Outlets Exposure ([modules/vpc/outputs.tf](#)):

```
# These export local IDs so outer parent files can dynamically see them
output "vpc_id" {
  value = aws_vpc.custom_network.id
}

output "subnet_id" {
  value = aws_subnet.public_subnet.id
}

output "subnet_id_b" {
  value = aws_subnet.public_subnet_b.id
}
```

4.3 Core Infrastructure, Firewall & Compute Automation ([main.tf](#))

This root configuration invokes your custom network layer, provisions security groups, and deploys your Jenkins and Ansible systems within the AWS Free Tier baseline.

```
# 1. Invoke local modular network infrastructure block
module "my_vpc_layer" {
  source = "../modules/vpc"
}

# 2. Firewall Group Security Policy Rules
resource "aws_security_group" "devops_sg" {
  name          = "vikas-devops-sg"
  description   = "Control perimeter transit security for deployment servers"
  vpc_id        = module.my_vpc_layer.vpc_id # Plugs cleanly into module outlet

  ingress {
    description = "Secure SSH remote administration management"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "Web interface interaction transit channel"
    from_port   = 8080
    to_port     = 8080
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port = 0
  }
}
```

```

    to_port      = 0
    protocol     = "-1"
    cidr_blocks  = ["0.0.0.0/0"]
  }
}

# 3. Provision Compute Nodes inside the Safe AWS Free-Tier Boundaries
resource "aws_instance" "jenkins_server" {
  ami              = "ami-007020fd9c84e18c7" # Ubuntu 24.04 LTS engine in Mumbai
  instance_type    = "t2.micro"                # Free tier eligible size
  subnet_id        = module.my_vpc_layer.subnet_id
  vpc_security_group_ids = [aws_security_group.devops_sg.id]

  tags = { Name = "Jenkins-Automation-Server" }
}

resource "aws_instance" "ansible_server" {
  ami              = "ami-007020fd9c84e18c7"
  instance_type    = "t2.micro"
  subnet_id        = module.my_vpc_layer.subnet_id
  vpc_security_group_ids = [aws_security_group.devops_sg.id]

  tags = { Name = "Ansible-Controller" }
}

```

4.4 Managed Infrastructure & Target Data Discovery ([rds.tf](#))

Demonstrates data blocks looking up your custom network environment dynamically to configure an RDS database subnet group and instance.

```

# Dynamic lookup filters searching your target infrastructure space
data "aws_vpc" "custom" {
  filter {
    name   = "tag:Name"
    values = ["vikas-vpc-01-vpc"] # Searches for your existing network text name
  }
}

data "aws_subnets" "custom" {
  filter {
    name   = "vpc-id"
    values = [data.aws_vpc.custom.id]
  }
}

resource "aws_db_subnet_group" "rds_subnets" {
  name          = "vikas-rds-subnet-group"
  subnet_ids    = data.aws_subnets.custom.ids
}

```

```

resource "aws_db_instance" "mysql_db" {
  allocated_storage    = 20
  engine               = "mysql"
  engine_version       = "8.0"
  instance_class       = "db.t4g.micro" # Cost-effective AWS Graviton instance
  db_name              = "my_beginner_db"
  username             = "admin"
  password             = "SuperSecurePass123!" # In production, use secret variables
  db_subnet_group_name = aws_db_subnet_group.rds_subnets.name
  skip_final_snapshot  = true
  publicly_accessible  = false
}

```

4.5 Output Reporting Structures (`outputs.tf`)

```

output "database_endpoint" {
  description = "Connection string for your application configurations"
  value       = aws_db_instance.mysql_db.endpoint
}

```

4.6 Containers & Advanced Cluster Pipelines (`containers.tf`)

```

# ECR Docker Image Registry Storage
resource "aws_ecr_repository" "docker_repo" {
  name                = "vikas-app-registry"
  image_tag_mutability = "MUTABLE"
  image_scanning_configuration { scan_on_push = true }
}

# IAM Role Mapping providing EKS administrative privileges
resource "aws_iam_role" "eks_cluster_role" {
  name = "vikas-eks-cluster-role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Action    = "sts:AssumeRole"
      Effect    = "Allow"
      Principal = { Service = "eks.amazonaws.com" }
    }]
  })
}

resource "aws_iam_role_policy_attachment" "eks_cluster_policy" {
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKSClusterPolicy"
  role       = aws_iam_role.eks_cluster_role.name
}

```



```

}

# High Availability EKS Cluster
resource "aws_eks_cluster" "k8s_cluster" {
  name      = "vikas-devops-eks-cluster"
  role_arn = aws_iam_role.eks_cluster_role.arn

  vpc_config {
    subnet_ids = [
      module.my_vpc_layer.subnet_id, # Zone 1a link
      module.my_vpc_layer.subnet_id_b # Zone 1b link
    ]
  }
  depends_on = [aws_iam_role_policy_attachment.eks_cluster_policy]
}

```

4.7 State Mapping Sandbox (`import_practice.tf`)

```

# Empty tracking shell that absorbs pre-existing resources from the web console
resource "aws_s3_bucket" "my_imported_bucket" {
  bucket = "vikas-terraform-state-1991"

  tags = {
    ManagedBy = "Terraform"
    Imported  = "22nd-May-Assignment"
    Owner     = "Vikas"
  }
}

```

CLI Import Syntax Tip: To activate the tracking block above, run this terminal script:

```
terraform import aws_s3_bucket.my_imported_bucket vikas-terraform-state-1991
```

5. Architecture Dependency and Core Concepts

To ensure your long-term success, let's look at how these files work together under the hood. There are three key concepts to master:

5.1 Implicit Resource Dependencies

Terraform doesn't read your files linearly from top to bottom. Instead, it parses all your code blocks simultaneously and builds a **Directed Acyclic Graph (DAG)** to map out dependencies. For example, look at this parameter on your servers:

```
subnet_id = module.my_vpc_layer.subnet_id
```

This tells Terraform that your EC2 instances have an *implicit dependency* on the VPC module. The engine recognizes that it must completely build the network and return its subnet ID output before it can trigger the server creation API. If your module has a syntax or configuration error, this dependency path blocks your servers from being planned or deployed.

5.2 Data Flows Across Folder Boundaries

By default, everything built inside a child module folder (like `modules/vpc/main.tf`) is isolated from the rest of your project. Think of a module like a secure building with no windows. To pass information between your modular network and your root configuration files, you must build explicit pipelines:

- An `output` block inside `modules/vpc/outputs.tf` installs a public data outlet on the outside wall of your module.
- A module reference assignment string in your root `main.tf` connects a configuration line directly to that public outlet.

5.3 The State File: Single Source of Truth

The `terraform.tfstate` JSON tracking document acts as the central brain of your architecture. It records a precise mapping of your HCL code blocks to actual real-world cloud resource IDs and attributes. When you run a plan, Terraform compares your local text configurations against the state file and your active cloud account to figure out exactly what changes are needed, preventing duplicate or overlapping deployments.

Security & Workflow Warning: The state file can contain sensitive configuration details, including database passwords in plaintext. Storing it securely in an encrypted cloud S3 bucket (Remote Backend) ensures your team stays synchronized, prevents deployment conflicts, and protects sensitive infrastructure data.